

Grouping `using` declarations with braces

Document #: P3485R0 [[Latest](#)] [[Status](#)]
Date: 2024-10-28
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction and Motivation](#)

[1.1 Workarounds](#)

[1.2 Other Language Support](#)

[1.3 Should we support `using \*`?](#)

[1.4 Should we support more than a single set of braces?](#)

[1.5 Where do the ellipses go?](#)

[1.6 Implementation Experience](#)

[2 Proposal](#)

[2.1 Wording](#)

[2.2 Feature-Test Macro](#)

[3 Acknowledgements](#)

§ 1 Introduction and Motivation

The goal of this paper is, in a nutshell:

Status Quo	Proposed
<pre>using std::chrono::duration; using std::chrono::time_point; using std::chrono::duration_cast;</pre>	<pre>using std::chrono::{duration, time_point, duration_cast};</pre>

This is purely an ergonomic benefit, this enables no new functionality.

The issue today is that when I want to bring names into scope, I have two options at my disposal. I could write:

```
using namespace std::chrono;
```

But this brings in an unknown amount of names, and technically not even into the current scope. It is frowned upon for good reason. Indeed, similar facilities in other languages are also frowned upon (such

as Python's from `module import *;`).

The other option is a using declaration for each name. This works fine, but when I'm bringing in lots of names from the same namespace, it is very repetitive in a way that offers no readability benefit:

```
using std::format;
using std::format_to;
using std::formatter;
using std::chrono::duration;
using std::chrono::time_point;
using std::chrono::duration_cast;
```

C++17's added the ability to group these together with commas (in order to support pack expansions in `using` declarations), which is a little better, in that it avoids having to repeat `using` and lets me group declarations:

```
using std::format, std::format_to, std::formatter;
using std::chrono::duration, std::chrono::time_point, std::chrono::duration_cast;
```

But this is still fairly repetitive, so it would be nice to not have to repeat the namespace in these groups. Even in the case of `std::`, that's just wasteful typing. But for long namespaces, and especially longer nested namespaces, it is a pretty decent readability improvement to just not have to do the repetition:

```
using std::{format, format_to, formatter};
using std::chrono::{duration, time_point, duration_cast};
```

§ 1.1 Workarounds

There are two workarounds I'm aware of for not having this feature.

One is to introduce a shorter name for the long namespace:

Status Quo	Workaround
<pre>using std::chrono::duration; using std::chrono::time_point; using std::chrono::duration_cast;</pre>	<pre>namespace C = std::chrono; using C::duration, C::time_point, C::duration_cast;</pre>

I don't really consider this much of a workaround. You have to introduce a new name, which you cannot un-introduce. The goal was to introduce `duration`, `time_point`, and `duration_cast`... not also `C`. Additionally while we achieve a terser declaration, we do so in a more cryptic way... which isn't much of a win. I would never use this.

Another is to introduce a macro like:

Status Quo	Workaround
<pre>using std::chrono::duration; using std::chrono::time_point; using std::chrono::duration_cast;</pre>	<pre>// using the C Preprocessor USING(std::chrono, (duration)(time_point) (duration_cast)); // using token sequence macros using!(^std::chrono, {"duration", "time_point", "duration_cast"});</pre>

These are close to the desired syntax and avoids the problem of introducing a new name for `std::chrono`. But uh... doesn't seem like an especially great substitute for a very simple language feature either.

§ 1.2 Other Language Support

Such a facility exists in other languages as well.

- Shell brace expansion works similar to as proposed here, except that it would swallow the comma. Nevertheless, it is a familiar enough syntax that many users will recognize the syntax and deduce the correct meaning from it.
- Rust using declarations, of the form `use std::collections::{BTreeSet, hash_map::{self, HashMap}};` as proposed here
- Scala likewise uses similar syntax, of the form `import scala.concurrent.{Future, Promise, blocking}`
- Python does not use this syntax, but does support a short-hand for importing several names from a module by way of `from a.b.c import x, y`, which is preferred to `from a.b.c import *`
- JavaScript/TypeScript is a mix of each, with the syntax `import {a, b} from "module"`
- The D language also has a way of importing a named list via `import std.stdio : writeln, readln;`
- Perhaps surprisingly, even C++ has existing practice here — but only for attributes: `[[using CC: opt(1), debug]]` is equivalent to `[[CC::opt(1), CC::debug]]`

§ 1.3 Should we support `using *`?

A follow-up question might be whether we should support `using std::chrono::*;` in addition to what I'm suggesting here. I don't think it's a good idea to do so.

For one thing, `using namespace std::chrono` already exists. For another, we would then have to ask if the globbing syntax means the same as the using-directive or not. It probably shouldn't. But I'd rather not even have to get into those questions, because I don't think it's a good idea in practice.

§ 1.4 Should we support more than a single set of braces?

Basically in addition to:

```
using std::chrono::{duration, time_point};
```

Should we also allow:

```
using std::{formatter, chrono::{duration, time_point}};
```

This isn't really any harder to implement than only allowing a single set of braces. But I don't think that personally I would ever write declarations with nested braces, while I definitely would declarations with one. For now, I'm only proposing one set of braces, not nested braces.

One single set of braces is probably at least 95% of the value of this feature. And, depending your view of the complexity of potential nested declarations, could be more than 100%.

§ 1.5 Where do the ellipses go?

If you want a using declaration that brings in two packs, do you write it like this:

```
using Ts::{as..., bs...};
```

Or like this:

```
using Ts::{as, bs}...;
```

The latter is shorter (don't have to repeat the `...s`), while the former is easier to implement (since you don't have to keep track of all the names while waiting for the `}`).

In practice, needing *two* such `using` declarations strikes me as exceedingly unlikely. And attempting to search for such usage on Github could only find a use in an [LLVM comment](#) introducing what a `UsingPackDecl` is.

I'll err on the side of just not supporting it, since it doesn't seem worthwhile. The vanishingly rare occurrence can just write it out the long way. Sorry to that one person.

§ 1.6 Implementation Experience

I implemented this [in clang](#), it was pretty straightforward. I'd estimate that somebody actually familiar with this codebase could have implemented it in 30 minutes.

The implementation allows arbitrary nested `{}`s, because it was easy to do. If we want to support that, that's okay, but I don't think I would ever write such a thing, so I'd rather just propose the more restricted set.

§ 2 Proposal

Extend the `using` declaration syntax from simply allowing a sequence of qualified-ids:

```
using    std::formatter,    std::format_to,    std::chrono::duration,  
        std::chrono::time_point;
```

To allowing at most one nested brace grouping.

```
// proposed OK  
using std::{formatter, format_to}, std::chrono::{duration, time_point};  
  
// proposed OK  
using std::{formatter, format_to};  
using std::chrono::{duration, time_point};  
  
// proposed OK  
using std::{formatter, format_to, chrono::duration, chrono::time_point};  
  
// proposed ill-formed  
using std::{formatter, format_to, chrono::{duration, time_point}};
```

§ 2.1 Wording

Change the grammar in 9.9 [\[namespace.udecl\]](#) to:

```
using-declaration:  
    using using-declarator-list ;  
  
using-declarator-list:  
- using-declarator ...opt  
- using-declarator-list , using-declarator ...opt  
+ using-declarator-elem  
+ using-declarator-list , using-declarator-elem  
  
+ using-declarator-elem:  
+ using-declarator ...opt  
+ typenameopt nested-name-specifier { possibly-qualified-id-list }  
  
+ possibly-qualified-id-list:  
+ nested-name-specifieropt unqualified-id  
+ possibly-qualified-id-list , nested-name-specifieropt unqualified-id  
  
using-declarator:  
    typenameopt nested-name-specifier unqualified-id
```

0 For the purposes of this clause, a *using-declarator-elem* of the form `typenameopt nested-name-specifier { possibly-qualified-id-list }` is equivalent to the sequence `typenameopt nested-name-specifier id1, typenameopt nested-name-specifier id2, ..., typenameopt nested-name-specifier idN`, where `idi` are the constituents of the *possibly-qualified-id-list*.

[*Example 1*:

```
namespace F::I {
    static constexpr int o = 1;
    static constexpr int n = 21;
    static constexpr int a = 2022;
}

using F::I::{o, n, a};           // equivalent to: using F::I::o,
                                F::I::n, F::I::a;
static_assert(o + n + a == 2044); // OK
```

— *end example*]

§ 2.2 Feature-Test Macro

Unnecessary. If you need to support older compilers, write the older code. There is no benefit to writing both.

§ 3 Acknowledgements

Thanks to John Filleau for originally [floating this idea](#) on the std-proposals list. Thanks to Tim Song for help with the edge cases.